Speaker Notes — Dissertation Defense

LLM-assisted Data Systems for Efficient ML Pipeline Generation and Query Optimization

Saeed Fathollahzadeh

Target: ∼40 min talk + Q&A

How to run the clock

Part	Slides	Budget	Leave at
0 — Why this thesis	1–9	8 min	8:00
1 — CatDB	10–16	11 min	19:00
2 — ReSequel	17–22	10 min	29:00
3 — ROLEX	23–25	6 min	35:00
4 — Summary and thanks	26–28	3 min	38:00

Two checkpoints. You should be starting **CatDB (slide 10)** at about 8 minutes and **ReSequel (slide 17)** at about 19 minutes. If you are late at either one, take the cut marked in red on the following slides — do not speed up your delivery.

One sentence to keep in your head. Every slide in this talk is either setting up or paying off the same claim: *an LLM is a good generator and a bad authority, so let it generate and let the data system verify.*

Part 0 — Why This Thesis (∼8 min)

Title slide [0:40]

Good morning and afternoon. Thank you all for being here, and thank you to my committee for taking the time to read this work.

I am Saeed, and I am defending my dissertation, "LLM-assisted Data Systems for Efficient ML Pipeline Generation and Query Optimization." This work was done under the supervision of Professor Essam Mansour at Concordia and Professor Matthias Boehm at TU Berlin.

I will speak for about forty minutes, and I would ask that we hold questions until the end — though if something is unclear on a slide, please stop me.

→ *Do not rush this. Breathe once before slide 1.*

Slide 1 Automating Data Systems: A Mainstay for Decades [1:00]

Let me start with some history, because this thesis is not a departure from what our field has always done — it is the next step in it.

Automating data systems is not new. Codd, in 1970, gave us the relational model, so that users could stop writing access paths by hand. Selinger, in 1979, gave us cost-based optimization, so that users could stop choosing join orders by hand. Starburst, in the nineties, gave us extensible rewrite rules. And on the machine-learning side, the same instinct produced auto-sklearn, and more recently a wave of LLM-based pipeline agents.

Fifty years of work, one recurring idea: **take a decision that requires an expert, and let the system make it instead.**

Slides 2–3 The Two Ecosystems — one narration across both [0:50]

So how well did that automation work? Look at what it produced — on both sides.

On the machine-learning side, an enormous ecosystem: scikit-learn, pandas, NumPy, PyTorch, TensorFlow, Hugging Face — and on top of them a second layer of AutoML and LLM-based pipeline builders. [Advance.] And on the database side, the same picture with different logos: PostgreSQL, MySQL, DuckDB, Spark, Snowflake — and behind them fifty years of optimizer research, from System R and Starburst through Cascades and Calcite.

Every one of these is a *building block*. **We automated the components on both sides. We did not automate what goes into them.**

On the left, nobody assembles the pipeline — choosing, cleaning, configuring, and gluing is still manual expert work. On the right, the optimizer does assemble the *plan* — that is what Selinger gave us — but the *query* is still written by hand, and the optimizer takes it as given. That is why the same data and the same semantics still vary by *orders of magnitude* in runtime, depending on how the query happens to be written.

→ Advance through slide 2 while still speaking; do not pause on it. Land on slide 3's red band and stop there.

→ Do not *say "we did not automate the assembly" — on the SQL side the optimizer* is *assembly automation, and a DB systems examiner will stop you on it. The distinction is plan vs. query, and it is what slide 17's 24% coverage result pays off.*

Slide 4 Example 1: Generate Pipelines for Unseen Datasets [1:20]

Let me make both problems concrete, starting with machine learning.

Here is a small salary dataset — a plain CSV. Experience, skills, gender, address, salary. The target is "Salary." The task is the one every data scientist starts with: build me a model that predicts it.

Now, what happens if I hand this to the two obvious automated approaches? I give AutoML the raw table, and it reaches **32.6%**. I give an LLM the schema and a prompt, and it reaches **39.2%**.

Look at why. "Experience" has values like "12 Months" and "two years" — the same thing written two ways. "Address" is a city and a postal code glued into one column. "Skills" is a list pretending to be a string. Neither the AutoML system nor the LLM knows any of this, because neither one *looked at the data*. They both guessed, and they both guessed wrong.

→ Point at "12 Months" and "two years" in the raw table. That single detail explains the entire CatDB contribution.

Slide 5 Example 2: Declarative Query Processing [1:20]

Now the same exercise for SQL. This is query 122 from the Stats benchmark — a real query, not one I constructed.

SQL is declarative: the user says *what*, the optimizer decides *how*. That is the promise. Here is what actually happens.

There is a `COUNT(*)` with no early aggregation, so the engine materializes an enormous intermediate. There is a flat four-way cross join. The join through `Users` is completely redundant — it is only a

primary-key bridge, it filters nothing. And the filters that could have cut the data early are applied *after* the join instead of before.

The result is on the right: roughly 3,800 seconds on MySQL, 3,500 on PostgreSQL, 92 on DuckDB. Same query, same answer, three engines, and all three are slow — because none of them rewrote what the user wrote.

→ *"The optimizer is not going to save you here" — say it out loud.*

Slide 6 Challenges for Data Systems Today — and the vision [0:55]

So let me put the two sides next to each other, because the parallel is exact.

For ML pipeline generation: data preparation dominates the effort; LLMs hallucinate on data they have never seen; and AutoML searches blindly and slowly.

For SQL optimization: runtime swings with the form of the query; hand-written rewrite rules are brittle; and any rewrite must stay provably equivalent.

Different communities, different vocabulary — but structurally the same problem.

[Advance.] Which gives the research vision for this thesis: **make LLMs a reliable component of data systems — generate like an expert, verify like a system.**

→ *The vision block appears on a click. Read it as written, pause, then go straight to slide 7 — do not paraphrase it, they will hear it again in the summary.*

Slide 7 My Research: LLM-assisted Data Systems [1:10]

Which brings me to the thesis statement.

The key idea is to pair the **creativity of LLMs** with the **verification power of data systems**. Generate candidates with the model; verify them with the system; keep only what is correct and fast.

I want to be precise about the division of labour, because this is the argument of the whole dissertation. The LLM is used as a *proposer*. It is never used as an *oracle*. Every claim the model makes is checked by running something.

Three systems apply that recipe. In **CatDB**, a data catalog grounds the prompt, and generated pipelines are executed and auto-repaired before they are accepted. In **ReSequel**, query templates steer the rewrite, and sample-based checking verifies equivalence before execution. In **ROLEX**, rewrite rules are discovered from the workload, verified once, and then reused across queries with no LLM in the loop at all.

→ *This is the spine. If a committee member remembers one slide, make it this one. Pause after "never used as an oracle."*

Slide 8 Preview: What LLM-assisted Data Systems Can Achieve [1:00]

Before the details, here is where we end up, on the two examples I just showed you.

On the salary dataset: AutoML got 32.6%, the LLM got 39.2%, and CatDB reaches **91.8%** — a 52.6% improvement in model performance, on the same data, with the same underlying model.

On query 122: ReSequel rewrites it to run **148 to 662 times** faster across DuckDB, MySQL, and PostgreSQL. ROLEX, which distills that rewrite into a reusable rule, gets **162 to 998 times**.

I will come back to each of these and show you how they are obtained.

→ *Let the numbers sit for two seconds. Do not editorialize.*

Slide 9 Thesis Contributions [1:00]

Here is the shape of the dissertation.

The first contribution is **CatDB**, published at PVLDB 2025, with a demonstration at SIGMOD 2025 — catalog-guided generation of data-centric ML pipelines. It builds on my earlier work, **GIO**, from SIGMOD 2023, on generating efficient readers for custom data formats. Together they cover the path from raw files to trained models.

The second is **ReSequel**, at PVLDB 2026 with a SIGMOD 2026 demonstration — an LLM rewrite layer that sits on top of any DBMS.

The third is **ROLEX**, currently under submission to SIGMOD 2027, which takes the rewrites ReSequel discovers and distills them into offline, verified, native rules — removing the LLM from the query path entirely.

One recipe across all of them: generate like an expert, verify like a system.

Short version: Read only the three system names and the venues; skip GIO. Saves 30 seconds.

Part 1 — CatDB (∼11 min)

CatDB paper cover [0:10]

Let me start with CatDB.

Slide 10 Motivation for CatDB [1:40]

This slide is the diagnosis, so let me walk it carefully. It is the same salary dataset, annotated with everything that goes wrong.

Look at the raw columns first. "Skills" is a sentence, not a plain string — `SQL, Java` is really a *list* feature, and it needs k-hot encoding, not string hashing. "Address" is composite — `7050 CA` is a zip and a state fused together, and it should be split into two features. "Gender" has "M" and "Male" and "F" and "Female" — semantically identical values that inflate the distinct count until the column stops looking categorical at all. And "Experience" has "36 months" and "3 Years," which are the same value.

Now look at what the LLM receives, in the middle. The metadata is noisy and incomplete — basic types only, no semantics. And the prompt is generic: it asks for a pipeline in general terms, with no per-stage, dataset-specific instruction.

So there are two failure modes stacked on top of each other. The model has never seen this dataset — it memorized Titanic, not your private data. And the description we hand it is wrong anyway.

The conclusion is at the bottom, and it is the founding idea of CatDB: **a data catalog stores exactly the profiling information that can precisely guide an LLM.** We already know how to compute it. We just were not giving it to the model.

→ *Slow down here. If they understand this slide, the rest of CatDB is mechanical.*

Slide 11 CatDB: LLM-based Generation of Data-centric ML Pipelines [1:30]

Here is the architecture, in three stages.

On the left, the **Data Catalog**. We profile the raw dataset — schema, feature types, distinct and missing value counts, basic statistics, column dependencies, and samples. We then refine that catalog: we correct feature types and deduplicate values, which I will show on the next slide.

In the middle, **Prompt Construction**. From the catalog we select the top-K features, project the metadata that matters, and attach explicit rules for each pipeline stage — preprocessing, feature engineering, model selection. If the dataset is small enough, that is a single prompt. If it is not, we split it into a *chain* of prompts, and I will come back to why.

On the right, **Manage Pipeline**. The LLM emits Python. We parse it, we execute it, and if it fails we repair it and re-execute.

The framing to keep is at the top: this is **zero-shot in-context learning**. No task-specific examples, no fine-tuning. The data catalog grounds the prompt, and execution verifies it.

Slide 12 CatDB: Schema Refinement [1:30]

Let me show you the refinement step, because it is where a surprising amount of the accuracy comes from.

Two problems. First, different representations of the same categorical value — the "12 Months" versus "two years" issue. Second, concatenated lists and composite features.

Our approach: identify the candidate string features, build a small prompt from the feature name and its distinct values, and let the LLM produce a *mapping* from the original values to a reduced set — and, at the same time, infer the correct feature type. That inference feeds back into the catalog, so it is iterative.

The table at the bottom is the result across fourteen columns. Look at the first one: **563 distinct values before, 100 after**. That is not just a tidy-up. A column with 563 distinct values does not look categorical to any downstream model; a column with 100 does. The refinement changes what the pipeline is even able to do with the feature.

→ *Point at the 563/100 pair. That is the number they will remember.*

Slide 13 CatDB: Prompt Construction and Pipeline Management [1:20]

Now the generation loop itself.

The prompt is assembled from rules plus schema in a template: a coding task for each stage, together with the refined catalog slice relevant to that stage. So the model is not asked "write me a pipeline"; it is asked to write a specific stage, given specific facts about specific columns.

Then we validate. We do a syntax check, and then we actually execute on the local dataset. And when it fails — which it does — we run a repair loop: we consult a local knowledge base of known errors and solutions, and where that is not enough, we go back to the LLM with the error message, the offending code, and the relevant catalog slice.

On the right you see the resulting pipeline DAG, and the payoff for the example: AutoML at 32.6%, a plain LLM at 39.2%, and CatDB at **91.8%**.

The important property here is that there are **no silent errors**. A pipeline either passes syntax and execution checks, or it does not get used. We never quietly accept something that looks plausible.

Slide 14 CatDB: Performance Comparison [1:10]

Now the evaluation, across three datasets — Etailing, Wifi, and Yelp — and against CAAFE with two backends, AutoGluon, H2O, Flaml, and AutoML with preprocessing.

Two bars in each group are ours: CatDB on the original data, and CatDB with the refined catalog. The refined version wins essentially everywhere, and the gap between our two bars is precisely the value of the catalog refinement.

The three mechanisms are listed underneath, and they map onto what I showed you: deduplication removes duplicate values and rebalances labels; categorical handling fixes formatting; and feature refinement drops constant and misread features while preserving the distribution.

Short version: Show only Etailing, name the mechanisms, move on. Saves 40 seconds.

Slide 15 CatDB: Cost and Runtime Comparison [1:10]

Accuracy is not the only axis — these systems cost money and time, so let me address that directly.

The top row is token count, the bottom row is runtime, across Diabetes, Gas-Drift, and Volkert, for three different LLMs.

Two findings. On cost: AIDE and AutoGen use hand-crafted prompts and consume substantially more tokens. CatDB consumes fewer, because we *project* the catalog — we send the metadata that matters for the stage, not everything we know. On runtime: CatDB is **8 to 14 times faster**, and I should be fair here — AIDE and AutoGen only count execution time in their reported numbers, so the real gap is larger than it looks.

So the grounding does not cost extra. It costs less, because a well-grounded prompt does not need to be long.

Slide 16 CatDB: Outlier and Missing Value Injection [1:20]

Finally, robustness — and this is the result I would defend hardest.

We inject missing values from 0 up to 50%, on top of 5% outliers, on two datasets: Utility, which is regression, and Volkert, which is multiclass.

CatDB is the red line at the top. It stays high across the whole sweep. Now look at the baselines: several of them do not merely degrade — they hit "Fail" on the axis. AutoSklearn fails immediately. Flaml degrades steadily. H2O and AutoGluon hold for a while and then drop.

The reason CatDB survives is not that the model is better. It is that the catalog *sees* the missing values and the outliers, and writes explicit handling instructions into the prompt for exactly those columns. The system knows what is wrong with the data before it asks the model for anything.

And this is at scale: twenty real-world datasets, up to 30 million rows, 478 columns, and up to 19 joined tables.

→ *Clock check — you want to be at about 19 minutes leaving this slide.*

Part 2 — ReSequel (∼10 min)

ReSequel paper cover [0:10]

That is the ML side. Let me turn to SQL.

Slide 17 Motivation for ReSequel: What Do Rewriters Actually Cover? [1:20]

I want to start by justifying that this problem is open, because the natural objection is "surely the optimizer already does this."

We took the twenty-one rewrite objectives that the literature identifies — redundancy elimination, access-path changes, aggregation pushdown, subquery handling, and so on — and asked, for each one, whether a given system actually covers it.

The DBMS optimizers cover **24%**. The non-DBMS rewrite tools cover **38%**. ReSequel covers **88%**.

I want to be careful about what that means. It is not that optimizers are badly built. It is that many of these rewrites require *semantic* knowledge — that a join is only a key bridge, that a filter is selective on this data — which the optimizer either does not have or cannot afford to derive at plan time. That is the gap an LLM can fill, because it reads the query the way a person does.

→ *Anticipate the objection out loud: "surely the optimizer does this already." Then answer it.*

Slide 18 ReSequel: LLM Rewriting as an Outer Optimization Layer [1:30]

Here is the design. The single most important thing about it is on the right: ReSequel is an **outer layer**. We do not touch engine internals. The same rewriter works against PostgreSQL, MySQL, and DuckDB.

Four design decisions follow from that.

It is **template-based**. We extract the query template and the parameters, mask the constants, and rewrite the *shape* once — then reuse it for every query with that shape. That is what makes it affordable.

It is **metadata-guided**. The prompt carries schema, statistics, inferred rules from the catalog, and rules by example — so, exactly the CatDB idea, transplanted to SQL.

It is **sample-verified**, which is the next slide.

And it is **cached**. On a cache hit, rewriting a query costs a lookup. The LLM cost is paid once per template and amortized over the workload.

Slide 19 Recap — Example 2 [0:40]

Let me bring back query 122 so the mechanism has something concrete to act on.

Recall the four problems: no early aggregation, a flat four-way cross join, a redundant join through `Users`, and filters applied after the join rather than before. Hold those in mind for the next slide.

Short version: Skip this slide entirely — click through it. Saves 40 seconds.

Slide 20 ReSequel: An Example Workflow [2:00]

Now watch the machine work on it, in four steps.

One: templatization. We take the query, mask the predicates — those dollar signs are the parameters — and we now have a template rather than one specific query. Every other query with this shape maps to the same template.

Two: the template rewriter. We prompt the LLM with the template plus the catalog rules, and we ask for N variants — not one. Ten of them here. This matters: we are not betting on the model getting it right once. We are sampling the space of rewrites.

Three: verification. We downsample the database to about 10%, run the original and each variant on that sample, and compare result sets. Any variant that disagrees is discarded, immediately. This is the step that makes LLM output safe to deploy.

Four: rank and cache. Among the survivors, we measure and take the fastest — that is the crowned bar, version 7 — and cache it against the template. Notice that version 10 is the original query, and notice that several variants are *slower* than the original. That is fine. We keep the original in the candidate set precisely so that we can never do worse than where we started.

The result for this query: factorized join, so `COUNT(*)` becomes a product of per-key counts and the flat join is never materialized; explicit joins on the CTEs instead of the four-way cross join; and filters pushed inside the CTEs. **148 to 662 times faster, same result.**

→ *This is the best slide in the talk. Give it the full two minutes and do not let the clock rush you through step three.*

Slide 21 ReSequel: End-to-End on Large Workloads [1:40]

Now, does that hold at workload scale? Here are StackOverflow and IMDB, on all three engines.

Every host improves — and I want to stress *every*, because this is not a single-engine result. PostgreSQL improves by 2.4× and 2.3×, MySQL by 4.8×, DuckDB by 3.9× and 1.4×.

Look at the bottom right, because that chart explains why this is affordable. StackOverflow has 1,190 queries but only **60 templates**. IMDB has 13,646 queries and **79 templates**. We pay the LLM 60 times and 79 times respectively — not 1,190 and 13,646 times. That ratio is the entire economic argument for template-level rewriting.

And note what is missing from these plots: the LLM baselines. R-Bot and LLM-R2 rewrite per query, and at this scale they simply cannot run.

Slide 22 ReSequel: End-to-End on Small Workloads [1:20]

The small workloads, Stats and Stats-CEB, where the baselines *can* run, so we can compare directly.

The speedups: Stats improves 5.89× on PostgreSQL and 14.85× on DuckDB; Stats-CEB improves 4×, 3.3×, and 16×. On MySQL, Stats is 1× — unchanged — and I would rather point that out myself than have you find it.

Where do the gains come from? Up to twenty variants per query, splitting data from catalog, and removing redundant `GROUP BY`s.

And the baselines: R-Bot rewrites only **12%** of the workload, and some of those rewrites have *negative* impact. LearnedRewrite is bound to Calcite's rule set — it improves few queries and often slows them down.

→ *Naming the 1× result yourself buys you credibility for the rest of the numbers.*

Part 3 — ROLEX (∼6 min)

ROLEX paper cover [0:10]

Which brings me to the last part, and to what I think is the most interesting question in the thesis.

Slide 23 Motivation for ROLEX: Where Should the LLM Live? [1:50]

ReSequel works. But look honestly at what it costs, on the left.

Today, the query is executed as written, and to improve it we make one LLM call per template. That means money, latency, and — worse — non-determinism: the same template can produce different rewrites on different days. The LLM emits free-form SQL, so the *insight* it found is thrown away the moment the query finishes. And verification happens per query, on samples.

So here is the question I want to pose: **where should the LLM live?**

ROLEX's answer is that it should live *offline*, and it should not emit queries at all — it should emit **rules**. We take a sample of the workload, ask the LLM for rewrites, distill the difference between original and rewrite into a general transformation rule, implement that rule as compiled C++, verify it once, and register it in the engine.

The four properties on the right are what that buys. It is **reproducible** — the same rewrite every run. It is **reusable** — one rule fires on many queries. It is **cheap** — zero online LLM cost. And it is **verified** — smoke test, then an SMT equivalence check, then a sample test, done once, offline, rather than per query.

We have moved the LLM off the hot path.

→ *"Where should the LLM live" is the thesis' best question. Ask it as a question and pause before answering.*

Slide 24 ROLEX: What the Rules Cost to Build [1:30]

The obvious objection is that we have not removed the cost, we have moved it. So let me quantify it.

The top plot is rule-generation time per workload: on the order of a few hundred seconds, once. The baselines — R-Bot, LLM-R2 — pay their cost *per query, at runtime*, every time. So the comparison is a one-time constant against a recurring charge.

Two more things worth noting. Distillation, generation, and verification all happen in memory, so the build cost is DBMS-independent. And across all workloads we generated 118 rules, which deduplicate to **60 distinct rules** — meaning workloads genuinely share structure, and the token cost tracks only the *new* rules a workload introduces.

By contrast, LLM-R2 needs per-workload re-training. Ours does not: the rule set carries over.

Slide 25 ROLEX vs. a Bespoke Query Compiler [1:40]

Let me close the technical part with the hardest comparison I could construct.

SynnoDB is a bespoke OLAP query compiler — it compiles each query down to native code over Parquet. This is not a fair fight in our favour; it is a specialized system built for exactly this.

Sixteen CEB queries over IMDB, scale factors 1 through 10.

ROLEX wins for a structural reason, not a tuning reason: it attacks the *asymptotic* cause. Factorizing the `COUNT(*)` means the huge intermediate is never built at all. On four of the queries that is 93% of DuckDB's total time, and the speedups there are in the hundreds. End to end, across all sixteen queries and every scale factor, we are about 1.8 to 2× over SynnoDB.

And look at the failure column, which I think is the real result: SynnoDB is the fastest system — until I/O and ingest and preprocessing are counted, at which point it is 58 to 77% of wall clock. Multi-threaded, it segfaults on two queries. DuckDB spills roughly 800 GB against 148 GB of RAM, and with 32 threads it is *slower* than with one, and four queries time out entirely.

ROLEX completes all sixteen, at every scale factor. A rewrite that removes the work beats an engine that does the work quickly.

→ *"A rewrite that removes the work beats an engine that does the work quickly" — that is the line. Deliver it and stop.*

Part 4 — Closing (∼3 min)

Slide 26 Summary [1:40]

Let me put the three systems in one frame.

One recipe, three systems: generate with an LLM, verify with the system, keep only what is correct and fast.

What grounds the LLM differs each time. In CatDB it is a data catalog plus a refined schema. In ReSequel it is query templates plus metadata. In ROLEX it is the workload itself.

What verifies differs too. CatDB executes and auto-repairs. ReSequel checks sample-based equivalence. ROLEX uses SMT plus sampling, once, offline.

And the headline results: up to 52% accuracy improvement; 2.5 to 16× per workload; and 60% below ReSequel with zero online LLM cost.

Three lessons I would draw from all of it. **Grounding beats prompting** — show the model the real data and the real queries, because it cannot guess them. **Verification is what makes LLM output deployable** — a rewrite that looks right can still be wrong, so always check before you use it. And **move the LLM off the hot path** — ask it once, keep what it found, and every later query is free.

That is the thesis.

→ *Do not add anything after "That is the thesis." Let it land.*

Acknowledgements divider [0:10]

[Advance. Say nothing.]

Slide 27 Acknowledgements — committee [0:30]

Before I take questions — thank you to my committee: Professor Carsten Binnig, Professor Juergen Rilling, Professor Abdelhak Bentaleb, and Professor Jun Yan. Thank you for the time you put into reading this.

Slide 28 Acknowledgements — supervisors [0:40]

And thank you, Essam and Matthias, for teaching me to ask better questions — and trusting me to answer them my own way.

Thank you all. I am happy to take your questions.

→ *Look up. Stop talking. Let the chair open the floor.*

Backup slides — what is where

The appendix sits *before* the acknowledgements in the deck, so you can jump into it and come back out without ending on a blank. It opens with an index slide listing the three systems.

Appendix slide	Use it when asked about
CatDB — prompt templates & chaining	how the prompt is actually built; why chaining exists; the token-limit argument
CatDB — datasets	scale of the evaluation; the 20 datasets, row and column ranges
CatDB — relation to KramaBench	how this differs from data-to-insight benchmarks; the "why not evaluate on KramaBench" question
CatDB — error management	the 23 error types, the KB/SE/RE split, the repair loop
CatDB — metadata impact	why more metadata is not better; the 260-feature collapse
ReSequel — workloads & datasets	the three workload groups; why templates pay off
ReSequel — rewriting cost	time breakdown; cache hits; what the LLM actually costs
ReSequel — downsampling sweep	why 10% is enough; validation overhead vs. ratio
ReSequel — ablation	sample sizes; Calcite vs. metadata vs. example rules
ROLEX — full architecture	the rule generator, fragment index, MCTS, three-phase verification

Questions to pre-stage

"Isn't this just prompt engineering?"
No — and the distinction is the thesis. Prompt engineering tunes wording against a fixed model. We change what the model is *shown*: computed, verified facts about this specific dataset or workload. And critically, we do not trust the output either way. Take away the catalog and CatDB drops to the plain-LLM number on slide 8.

"What happens when the LLM is wrong?"
It is wrong often, and the systems are built for that. CatDB: the pipeline fails to parse or execute, and the repair loop runs — appendix slide has the 23 error types and their distribution. ReSequel: the variant disagrees with the original on the sample and is discarded; the original is always in the candidate set, so the floor is "no change." ROLEX: the rule fails smoke, SMT, or sample verification and never enters the rule set.

"Why sampling instead of formal verification?"
We use both, in ROLEX, in that order — SMT first, sampling only when the solver returns UNKNOWN. Formal verifiers cannot decide most real LLM-generated SQL (date arithmetic, complex aggregates) and can take days on a full workload. Sampling is the fallback that keeps the guarantee practical, and a single mismatch rejects the rule.

"Does this generalize beyond the datasets you tried?"
Twenty datasets for CatDB, up to 30M rows and 478 columns; eight benchmarks and three engines for ReSequel. The stronger evidence is ROLEX's deduplication: 118 rules collapse to 60 distinct ones across workloads, which says the structure being exploited is shared, not per-dataset.

"What is the cost, in dollars?"
CatDB uses fewer tokens than hand-prompted baselines because it projects the catalog (slide 15). ReSequel pays once per template — 60 calls for 1,190 StackOverflow queries. ROLEX pays once per workload, offline, and nothing at query time.

"Why three systems instead of one?"
They are one system's idea at three points on a spectrum of how much LLM you keep at runtime. CatDB: LLM at generation time, verified by execution. ReSequel: LLM at template time, verified by sampling, cached. ROLEX: LLM entirely offline, verified once, compiled away. The trajectory — pushing the model further from the hot path as verification gets stronger — is the argument.

“What would you do next?”
Two things. Close the loop between ROLEX’s rule set and the optimizer’s cost model, so rule ordering is cost-driven rather than MCTS-driven. And extend CatDB past supervised learning — unsupervised and time-series pipelines, where the held-out label is no longer available as a free oracle.